

Relatório Técnico — Padrões Estruturais: Adapter & Decorator

Disciplina: SI405 — Engenharia de Software / Padrões de Projeto

Atividade: AP6 — Padrões Estruturais

Autor: Bruno César de Souza Ferreira França

RA: 167955

Data: 07 de julho de 2026

Repositório: github.com/thebrunofranca/SI405-AP6

1. Análise dos Problemas Arquiteturais Identificados

1.1 Acoplamento com implementações concretas de gateways

O `CobrancaService` instanciava `GatewayPagamentoInterno` e `PaySecureGateway` diretamente dentro dos seus métodos, violando o princípio de inversão de dependência. Qualquer adição de um novo gateway exigia modificação direta no serviço de cobrança.

1.2 Duplicação da lógica de seleção e adaptação de gateway

Os métodos `cobrar` e `cobrarEmLote` continham blocos `if/else` idênticos para seleção e invocação do gateway, incluindo todo o código de construção do `Map`, chamada ao SDK e tratamento de exceções. A duplicação tornava a manutenção propensa a erros.

1.3 Interfaces incompatíveis entre gateways

`GatewayPagamentoInterno` expõe `cobrar(String, String, double, FormaPagamento)` enquanto `PaySecureGateway` expõe `processarTransacao(Map<String, Object>)` lançando `GatewayIndisponivelException`. O `WalletPaySDK` por sua vez usa `charge(ChargeRequest)` com valores em centavos. A ausência de uma abstração comum forçava o serviço a conhecer os detalhes de cada API.

1.4 Ajustes de valor controlados por parâmetros booleanos

O método `calcularValorFinal` recebia quatro parâmetros booleanos. Essa abordagem viola o princípio Aberto/Fechado: adicionar qualquer novo ajuste obrigava a alterar a assinatura do método e atualizar todos os chamadores. A combinação de ajustes também ficava implícita em vez de explícita.

2. Justificativa das Decisões Arquiteturais Adotadas

Adapter sobre herança ou modificação direta

Os SDKs externos (`PaySecureGateway` , `WalletPaySDK`) representam bibliotecas de terceiros que não podem ser modificadas. O padrão Adapter permite adaptá-los sem tocar no código original, isolando a lógica de conversão em classes dedicadas.

Interface `GatewayCobranca` como ponto de extensão

A criação de uma interface comum permite que o `CobrancaService` trabalhe apenas com a abstração, ignorando completamente os detalhes de cada gateway. A adição de novos meios de pagamento requer apenas a criação de um novo adapter, sem alterar o serviço.

Decorator com varargs em vez de parâmetros booleanos

A substituição dos booleanos por instâncias de `AjusteValor` passadas como varargs elimina a explosão combinatória de parâmetros. Cada ajuste é independente, combinável em qualquer ordem, e novos ajustes são adicionados sem nenhuma alteração nas assinaturas existentes.

`GatewayPagamentoInternoAdapter` como wrapper

Embora `GatewayPagamentoInterno` seja código próprio, optou-se por envolvê-lo em um adapter ao invés de modificar sua interface, mantendo sua testabilidade independente e respeitando o princípio da responsabilidade única.

3. Aplicação do Padrão Adapter

Estrutura implementada

```
GatewayCobranca (interface)
├── GatewayPagamentoInternoAdapter → wraps GatewayPagamentoInterno
├── PaySecureGatewayAdapter         → wraps PaySecureGateway (externo)
└── WalletPaySDKAdapter             → wraps WalletPaySDK      (externo)
```

Interface alvo

```
public interface GatewayCobranca {
    ResultadoCobranca cobrar(Pedido pedido, double valor, FormaPagamento forma);
}
```

Exemplo — `PaySecureGatewayAdapter`

O adapter traduz a chamada uniforme `cobrar(pedido, valor, forma)` para a interface proprietária do `PaySecureGateway`: constrói o `Map<String, Object>`, chama `processarTransacao`, converte o código de status em `"APROVADA"` / `"RECUSADA"` e absorve a `GatewayIndisponivelException`.

Exemplo — `WalletPaySDKAdapter`

Converte o valor em reais para centavos (`Math.round(valor * 100)`), constrói um `ChargeRequest`, chama `sdk.charge()` e converte `ChargeStatus.CONFIRMED` para `"APROVADA"`.

Novo meio de pagamento: Carteira Digital

O `FormaPagamento` recebeu a constante `CARTEIRA_DIGITAL`. O `CobrancaService` mapeia essa forma para o `WalletPaySDKAdapter` no seu método `selecionarGateway`. O `Main` expõe a opção 4 no menu interativo.

Impacto no `CobrancaService`

O serviço passou a depender apenas de `GatewayCobranca`. A seleção do gateway ocorre uma única vez, em `selecionarGateway(FormaPagamento)`, eliminando toda a duplicação entre `cobrar` e `cobrarEmLote`.

4. Application do Padrão Decorator

Estrutura implementada

```
AjusteValor (interface)
├── DescontoFidelidadeDecorator      (-5%)
├── JurosParcelamentoDecorator      (+2,99%)
├── TaxaInternacionalDecorator      (+5%)
├── SeguroDecorator                  (+R$ 4,90)
├── TaxaAntecipacaoReceiveisDecorator (+1,5%) ← novo
└── TaxaEmissaoNotaFiscalDecorator  (+R$ 2,50) ← novo
```

Interface componente

```
public interface AjusteValor {
    double aplicar(double valor);
}
```

Cada implementação aplica sua transformação sobre o valor recebido e retorna o resultado. Os decorators são compostos sequencialmente pelo `CobrancaService`:

```
public double calcularValorFinal(double valorBase, AjusteValor... ajustes) {
    double valor = valorBase;
    for (AjusteValor ajuste : ajustes) {
        valor = ajuste.aplicar(valor);
    }
    return valor;
}
```

Nova assinatura dos métodos de cobrança

```
public ResultadoCobranca cobrar(Pedido pedido, FormaPagamento forma, AjusteValor... ajustes)
public List<ResultadoCobranca> cobrarEmLote(List<Pedido> pedidos, FormaPagamento forma, AjusteValor... ajustes)
```

A combinação de ajustes é declarativa no ponto de chamada:

```
service.cobrar(pedido, FormaPagamento.BOLETO,
    new DescontoFidelidadeDecorator(),
    new TaxaAntecipacaoReceiveisDecorator(),
    new TaxaEmissaoNotaFiscalDecorator());
```

Novos ajustes adicionados

Ajuste	Tipo	Valor
<code>TaxaAntecipacaoReceiveisDecorator</code>	Percentual	+1,5% sobre o valor
<code>TaxaEmissaoNotaFiscalDecorator</code>	Fixo	+R\$ 2,50

5. Impactos da Refatoração

Organização

O código foi reorganizado em pacotes com responsabilidades claras:

- `gateway/` — abstração e adapters de meios de pagamento
- `ajuste/` — decorators de ajuste de valor
- `service/` — orquestração, agora sem dependências concretas externas

Manutenção

- Falhas em um gateway não afetam os demais: cada adapter trata suas próprias exceções de forma isolada.
- A lógica de conversão de cada SDK está contida em uma única classe, facilitando debugging e atualização.

- A remoção dos parâmetros booleanos eliminou o risco de erro por inversão de argumentos (`true` / `false` na ordem errada).

Extensibilidade

- **Novo gateway:** criar uma classe que implemente `GatewayCobranca` e adicionar uma entrada no `switch` de `selecionarGateway`. Nenhuma outra classe precisa mudar.
- **Novo ajuste de valor:** criar uma classe que implemente `AjusteValor`. Nenhuma assinatura de método existente precisa mudar.
- A arquitetura passa a respeitar o princípio Aberto/Fechado: aberta para extensão, fechada para modificação.

Testes

Suite de testes	Antes	Depois
<code>CobrancaServiceTest</code>	13 testes (booleanos)	14 testes (decorators + CARTEIRA_DIGITAL)
<code>GatewayPagamentoInternoTest</code>	2 testes	2 testes (inalterados)
<code>PaySecureGatewayTest</code>	4 testes	4 testes (inalterados)
<code>PedidoTest</code>	3 testes	3 testes (inalterados)
<code>WalletPaySDKTest</code>	3 testes	3 testes (inalterados)
<code>PaySecureGatewayAdapterTest</code>	—	4 testes (novo)
<code>WalletPaySDKAdapterTest</code>	—	5 testes (novo)
<code>AjusteValorDecoratorTest</code>	—	13 testes (novo)
Total	25	48

Todos os 48 testes passam com `BUILD SUCCESS`.